

Process & Decision Documentation

Side Quests and A4 (Individual Work)

Keep this section brief, typically 2 to 4 sentences.

Focus on:

- One significant decision or change you made
- Why you made it
- What effect it had on the work

Examples:

- Simplifying a mechanic so it functioned correctly
- Changing an approach after something failed
- Deciding not to pursue an idea due to time or technical limitations

You are not expected to document every alternative or iteration

- I decided to make changes to the example 3 code to make the blob game feel more joyful
- I decided to make the speed faster, gravity lower, and jumps higher to try and exude that emotion
- I also decided to change the environment colours to make them fit the emotions

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used:

Tool Disclosure: ChatGPT 5.2

Purpose of Use: I used GenAI to generate the new code for my redesign

Summary of Interaction:

- I prompted it by asking questions on how to code a specific feature of the game such as changing the size of the ball, changing the speed and height of each jump of the ball to better showcase the emotion
- I also asked it initially to give me ideas on how I can redesign the blob for different emotions: “how can I redesign the blob’s movement and environment to express a specific emotion (e.g., joy, frustration, panic)?”
- Afterwards, I asked it to give me specific codes on things I liked such as changing the speed or jumping effects and the colours and sizes
- I also had to do some trouble shooting at times when the code did not work for me and either change directions or modify it to make it work

Prompts:

- “can you tell me what this code creates?” inserted code*
- “how can I redesign the blob’s movement and environment to express a specific emotion (e.g., joy, frustration, panic)?”
- “how do I change the background colour to yellow”
- “how do i make the blob smaller”
- “the blob jumps higher with lower gravity and everything but jumps vertically off the screen”

Human Decision Point(s): Describe where you overruled, modified, rejected, or redirected the GenAI output, and why.

Integrity & Verification Note: To ensure that the GenAI would give me the correct code that aligned with the course content, the first thing I did was paste the whole original code from the example and asked it what the output would look like. This helped me verify if it could accurately decipher the code and then help me generate new code within that scope.

Scope of GenAI Use: GenAI gave me the correct ideas and lines of codes but I modified most of them to fit my interests such as picking a different colour or changing the speed or heights to my desires

Limitations or Misfires: I had trouble with the tool when trying to add new things to the code such as different jumping effects, after testing the code at times it would not work

Appendix

```
can you tell me what this code creates: // Y-position of the floor (ground level) let floorY3; //
Player character (soft, animated blob) let blob3 = { // Position (centre of the blob) x: 80, y: 0, //
Visual properties r: 26, // Base radius points: 48, // Number of points used to draw the blob
wobble: 7, // Edge deformation amount wobbleFreq: 0.9, // Time values for breathing animation
t: 0, tSpeed: 0.01, // Physics: velocity vx: 0, // Horizontal velocity vy: 0, // Vertical velocity //
Movement tuning accel: 0.55, // Horizontal acceleration maxRun: 4.0, // Maximum horizontal
speed gravity: 0.65, // Downward force jumpV: -11.0, // Initial jump impulse // State onGround:
false, // True when standing on a platform // Friction frictionAir: 0.995, // Light friction in air
frictionGround: 0.88, // Stronger friction on ground }; // List of solid platforms the blob can
stand on // Each platform is an axis-aligned rectangle (AABB) let platforms = []; function setup()
{ createCanvas(640, 360); // Define the floor height floorY3 = height - 36; noStroke();
textFont("sans-serif"); textSize(14); // Create platforms (floor + steps) platforms = [ { x: 0, y:
floorY3, w: width, h: height - floorY3 }, // floor { x: 120, y: floorY3 - 70, w: 120, h: 12 }, // low
step { x: 300, y: floorY3 - 120, w: 90, h: 12 }, // mid step { x: 440, y: floorY3 - 180, w: 130, h:
12 }, // high step { x: 520, y: floorY3 - 70, w: 90, h: 12 }, // return ramp ]; // Start the blob resting
on the floor blob3.y = floorY3 - blob3.r - 1; } function draw() { background(240); // --- Draw all
platforms --- fill(200); for (const p of platforms) { rect(p.x, p.y, p.w, p.h); } // --- Input: left/right
movement --- let move = 0; if (keyIsDown(65) || keyIsDown(LEFT_ARROW)) move -= 1; // A
or ← if (keyIsDown(68) || keyIsDown(RIGHT_ARROW)) move += 1; // D or → blob3.vx +=
blob3.accel * move; // --- Apply friction and clamp speed --- blob3.vx *= blob3.onGround ?
blob3.frictionGround : blob3.frictionAir; blob3.vx = constrain(blob3.vx, -blob3.maxRun,
blob3.maxRun); // --- Apply gravity --- blob3.vy += blob3.gravity; // --- Collision representation
--- // We collide using a rectangle (AABB), // even though the blob is drawn as a circle let box =
{ x: blob3.x - blob3.r, y: blob3.y - blob3.r, w: blob3.r * 2, h: blob3.r * 2, }; // --- STEP 1: Move
horizontally, then resolve X collisions --- box.x += blob3.vx; for (const s of platforms) { if
(overlap(box, s)) { if (blob3.vx > 0) { // Moving right → hit the left side of a platform box.x =
s.x - box.w; } else if (blob3.vx < 0) { // Moving left → hit the right side of a platform box.x = s.x
+ s.w; } blob3.vx = 0; } } // --- STEP 2: Move vertically, then resolve Y collisions --- box.y +=
blob3.vy; blob3.onGround = false; for (const s of platforms) { if (overlap(box, s)) { if (blob3.vy
> 0) { // Falling → land on top of a platform box.y = s.y - box.h; blob3.vy = 0; blob3.onGround
= true; } else if (blob3.vy < 0) { // Rising → hit the underside of a platform box.y = s.y + s.h;
blob3.vy = 0; } } } // --- Convert collision box back to blob centre --- blob3.x = box.x + box.w /
```

```

2; blob3.y = box.y + box.h / 2; // Keep blob inside the canvas horizontally blob3.x =
constrain(blob3.x, blob3.r, width - blob3.r); // --- Draw the animated blob --- blob3.t +=
blob3.tSpeed; drawBlobCircle(blob3); // --- HUD --- fill(0); text("Move: A/D or ←/→ • Jump:
Space/W/↑ • Land on platforms", 10, 18); } // Axis-Aligned Bounding Box (AABB) overlap test
// Returns true if rectangles a and b intersect function overlap(a, b) { return ( a.x < b.x + b.w &&
a.x + a.w > b.x && a.y < b.y + b.h && a.y + a.h > b.y ); } // Draws the blob using Perlin noise
for a soft, breathing effect function drawBlobCircle(b) { fill(20, 120, 255); beginShape(); for (let
i = 0; i < b.points; i++) { const a = (i / b.points) * TAU; // Noise-based radius offset const n =
noise( cos(a) * b.wobbleFreq + 100, sin(a) * b.wobbleFreq + 100, b.t, ); const r = b.r + map(n, 0,
1, -b.wobble, b.wobble); vertex(b.x + cos(a) * r, b.y + sin(a) * r); } endShape(CLOSE); } // Jump
input (only allowed when grounded) function keyPressed() { if ( (key === " " || key === "W" ||
key === "w" || keyCode === UP_ARROW) && blob3.onGround ) { blob3.vy = blob3.jumpV;
blob3.onGround = false; } } /* In-class tweaks for experimentation: • Add a new platform:
platforms.push({ x: 220, y: floorY3 - 150, w: 80, h: 12 }); • "Ice" feel → frictionGround = 0.95 •
"Sand" feel → frictionGround = 0.80 */

```

Yep — this is a **small 2D platformer demo made in p5.js** with a soft, animated “blob” character. Here’s what it creates, piece by piece, in plain English 📌

What you’d see on screen

- A **640 × 360 canvas**
- A **light grey background**
- Several **flat rectangular platforms** forming:
 - A ground floor
 - A few step-like platforms at increasing heights
- A **blue, jelly-like blob** character that:
 - Wobbles and “breathes”
 - Can run, jump, fall, and land on platforms

Think **Mario-style platformer**, but with a **cute, squishy physics blob** instead of a rigid sprite.

The player character (the blob)

Visually:

- Drawn as a **circle made of many points**
- Uses **Perlin noise** to deform its edge
- Constant subtle wobble → feels alive / organic

Movement & physics:

- **Left / Right movement**
- **Gravity** pulls it down
- **Jumping** only when touching the ground
- Different **friction in air vs on ground**
- Max run speed so it doesn't go crazy

Controls:

- Move: **A / D** or $\leftarrow$ / $\rightarrow$
- Jump: **Space / W** / $\uparrow$

Platforms & collision

- Platforms are **rectangles (AABBs)** stored in an array
- The blob *looks* circular, but **collides as a box** (simpler + faster)
- Collision handling is split into:
 1. **Horizontal movement + collision**
 2. **Vertical movement + collision**

This prevents common bugs like:

- Getting stuck in walls
- Falling through platforms
- Sticky corners

Landing logic:

- If falling and touching a platform $\rightarrow$ **snap to top + set `onGround = true`**
- If jumping up into a platform $\rightarrow$ **bonk head, stop upward motion**

Animation magic ✨

The wobble comes from this idea:

- Loop around the circle
- For each angle:
 - Sample **Perlin noise**
 - Slightly increase or decrease radius
- Over time (t), the noise changes $\rightarrow$ smooth breathing motion

Result:

A soft, living blob that feels way more playful than a normal circle.

HUD

At the top:

Move: A/D or ←/→ • Jump: Space/W/↑ • Land on platforms

What this code is *really* good for

This is a **teaching / prototyping platformer**:

- Intro to **game physics**
 - Clean **collision resolution**
 - Visual feedback that feels fun
 - Easy to tweak for:
 - Ice levels
 - Sand levels
 - New platforms
 - Different jump feels
-

One-sentence summary

This code creates a **playable 2D platformer prototype** where you control a **soft, wobbling blob** that runs and jumps across platforms using **real-time physics, collision detection, and organic animation**.

If you want, I can:

- Turn this into a **camera-follow level**
- Add **double jump / wall jump**
- Add **enemies or collectibles**
- Explain it like you're a **design student** vs **game dev student**

Just say the word 😊

how can I redesign the blob's movement and environment to express a specific emotion (e.g., joy, frustration, panic)?

This is a *great* question — you're basically asking how to turn **game feel into emotional storytelling**. I'll break this down in a **designer-friendly way**, not just code tweaks.

Think of emotion as coming from **three layers working together**:

1. **Movement feel (physics)**
2. **Animation behavior (how the blob deforms/reacts)**
3. **Environment response (how the world treats the blob)**

I'll show you how *joy*, *frustration*, and *panic* would feel different across those layers 📌

JOY — light, playful, bouncy

Movement

Joy feels **effortless and forgiving**.

- Low gravity → floaty jumps
- High jump impulse
- Strong air control
- Elastic rebounds

Tweaks

```
gravity: 0.45,  
jumpV: -13,  
accel: 0.7,  
frictionGround: 0.9,  
maxRun: 5.5
```

Add **bounce on landing**:

```
if (blob3.onGround && abs(prevVy) > 4) {  
  blob3.vy = -abs(prevVy) * 0.3;  
}
```

Animation

- Bigger wobble
- Faster breathing
- Slight squash on landing

```
wobble: 10,  
tSpeed: 0.02
```

Design principle:

Joy = excess motion. Nothing settles immediately.

Environment

- Rounded platforms
- Spring pads
- Upward slopes
- Bright colors

Emotion trick:

Let the world *help* the player.

❏ FRUSTRATION — heavy, sticky, resistant

Movement

Frustration feels like **fighting the controls** (but not broken).

- Strong gravity
- Weak jump
- Sluggish acceleration
- High ground friction

```
gravity: 0.9,  
jumpV: -9,  
accel: 0.35,  
frictionGround: 0.75,  
maxRun: 3
```

Add **input delay or inertia**:

```
blob3.vx += blob3.accel * move * 0.7;
```

Animation

- Reduced wobble
- Slower breathing
- Slight asymmetry in shape

```
wobble: 4,  
tSpeed: 0.006
```

Design idea:

The blob looks tired, like it doesn't *want* to move.

Environment

- Narrow platforms
- Awkward gaps
- Low ceilings
- Sticky surfaces (mud)

Emotion trick:

The environment resists progress.

3 PANIC — unstable, twitchy, out of control 🥵

Movement

Panic is **too much energy**.

- High gravity
- Very strong jump
- Slippery ground
- Over-acceleration

```
gravity: 1.2,  
jumpV: -15,  
accel: 1.1,  
frictionGround: 0.95,  
maxRun: 7
```

Add **micro jitters**:

```
blob3.vx += random(-0.1, 0.1);
```

Animation

- Fast, sharp wobble
- Uneven deformation
- Irregular breathing speed

```
wobble: 14,  
tSpeed: random(0.02, 0.05)
```

You can even distort vertically more than horizontally.

Environment

- Falling platforms
- Tight timing windows
- Alarms / flashing colors
- Slanted or moving ground

Emotion trick:

The world feels like it's collapsing faster than you can react.

how do I change the background colour to yellow

Super easy 👍 — you already have the right hook in place.

Right now your background is set here in `draw()`:

```
background(240);
```

Option 1: Simple yellow (recommended)

Replace it with:

```
background(255, 235, 80);
```

That gives you a warm, joyful yellow (nice choice for the emotion you're exploring ☀️).

Option 2: Bright pure yellow

```
background(255, 255, 0);
```

Very loud, very energetic — can feel playful *or* overwhelming depending on the vibe.

Option 3: Soft pastel yellow (more UX-friendly)

```
background(255, 245, 200);
```

Good if you want joy without eye strain.

Pro tip (design thinking)

For **joy**, avoid max saturation. Slightly muted yellows feel:

- friendlier
- calmer
- more “happy” than “warning sign”

If you want, next we can:

- Animate the background brightness with the blob’s bounce
- Shift color based on emotion state (joy → panic)
- Match platform colors to emotional tone

Just tell me what you’re aiming for 🎨✨

the blob jumps higher with lower gravity and everything but jumps vertically off the screen

Yep — that’s a **classic side-effect of “joyful” tuning** 😅

Lower gravity + strong jump = *nothing pulls the blob back down fast enough*, so it rockets off-screen.

The fix isn’t “undo joy” — it’s **control the jump apex**.

Here are the **clean, correct ways** to fix it (from most to least designer-friendly).



Best fix: clamp upward velocity

Right after you apply the jump impulse in `keyPressed()`:

```
blob3.vy = blob3.jumpV;  
blob3.vy = max(blob3.vy, -12);
```

Or more cleanly:

```
const MAX_JUMP_UP = -12;  
blob3.vy = max(blob3.jumpV, MAX_JUMP_UP);
```

This keeps jumps floaty but **caps the launch power**.

✓ Even better: shorten jump while the key is released (variable jump height)

This is how *Nintendo platformers* do it.

Add this in `draw()`:

```
if (
  blob3.vy < 0 &&
  !keyIsDown(32) &&
  !keyIsDown(87) &&
  !keyIsDown(UP_ARROW)
) {
  blob3.vy *= 0.6;
}
```

What this does:

- Hold jump → high jump
- Tap jump → short hop
- Prevents accidental moon launches 🚀

✓ Add a soft ceiling (failsafe)

After vertical movement:

```
blob3.y = max(blob3.y, blob3.r);
```

This keeps the blob from leaving the canvas *ever*.

✗ What *not* to do

- Don't just increase gravity again → kills floatiness
- Don't remove bounce → removes joy
- Don't clamp `y` before collision resolution → causes snapping bugs

🎯 Recommended joyful tuning combo

```
gravity: 0.45,
jumpV: -13,
MAX_JUMP_UP = -12
```

Plus:

- Variable jump height
- Bounce on landing
- Soft ceiling

Design takeaway (this is critique-ready)

Joy is about forgiveness, not infinite freedom.

Players should feel powerful — but never lose spatial awareness.

If you want, I can:

- Tune jump height to match **exact platform distances**
- Add **camera follow with easing**
- Visualize jump arcs for playtesting

You're asking *exactly* the right questions — keep going 🙌

how do i make the blob smaller

Nice and simple one — and you've already got it set up perfectly for this 👍

The one value that controls blob size

In your blob object:

```
r: 26, // Base radius
```

That's it.

Make this number smaller.

Example

```
r: 18
```

or even cuter:

```
r: 14
```

⚠️ Important things to remember (so nothing breaks)

Because the blob uses `r` for **everything** (drawing *and* collision), changing it affects:

- Collision box size
- How high it sits on platforms
- Jump clearance under ceilings

So after changing `r`, do **one more small fix** 📌

Re-position the blob on the floor in `setup()`

You already have this line:

```
blob3.y = floorY3 - blob3.r - 1;
```

That automatically adapts to the new size — so you're good ✅

No extra work needed.

Optional: scale the wobble with size (recommended)

If you make the blob smaller but keep the same wobble, it can look *too blobby*.

Try this:

```
wobble: 4
```

or dynamically:

```
wobble: blob3.r * 0.25
```

Designer tip 🎨

Smaller blob = reads as:

- lighter
- faster
- more playful
- more “childlike”

If you're going for **joy**, a slightly smaller blob + higher bounce actually reinforces the emotion.

If you want next:

- Animate size based on emotion (panic → shrink)

- Squash/stretch without changing collision size
- Make size change over time like breathing

Just tell me what vibe you're chasing 😊

this works but the blob gets a little sticky on the top: `blob3.y = max(blob3.y, blob3.r);`

Yep — you're not imagining it.

That “**sticky ceiling**” feeling is a *very real* side-effect of that line 👍

Why it happens (quick + clear)

This line:

```
blob3.y = max(blob3.y, blob3.r);
```

forces the blob's center to sit exactly at `y = blob3.r` when it reaches the top.

But when the blob is still moving upward:

- `vy` is **negative**
- You clamp the position
- **Velocity stays negative**
- Next frame it tries to move up again → clamp again

Result:

👉 the blob *slides and sticks* to the ceiling instead of bouncing or falling.

✅ The correct fix: cancel upward velocity when hitting the ceiling

You want **position clamp + velocity response**, just like a real collision.

Replace your clamp with this:

```
if (blob3.y < blob3.r) {
    blob3.y = blob3.r;

    if (blob3.vy < 0) {
        blob3.vy = 0;
    }
}
```

Why this works

- Stops upward motion immediately
- Gravity takes over next frame

- No stickiness
- Feels intentional, not hacked

p5.js green colour

In **p5.js**, colors can be specified using **RGB values**, **hex codes**, or the built-in `color()` function. For **green**, here are some common ways to use it:

1 Using `fill()` or `stroke()` with RGB

```
function setup() {
  createCanvas(400, 400);
  background(220);

  fill(0, 255, 0); // bright green
  rect(50, 50, 100, 100);

  stroke(0, 128, 0); // dark green outline
  strokeWeight(4);
  rect(200, 50, 100, 100);
}
```

- `fill(r, g, b)` → fills shapes with color
- `stroke(r, g, b)` → sets outline color
- RGB values range from 0 to 255

light blue

In **p5.js**, “light blue” can be represented using **RGB values**, **hex codes**, or **HSB**. Here are the common ways:

Using RGB

```
function setup() {
  createCanvas(400, 400);
  background(220);

  fill(173, 216, 230); // light blue
  rect(50, 50, 300, 300);
}
```